

Efficient Depth Estimation for Unstable Stereo Camera Systems on AR Glasses

Yongfan Liu Hyoukjun Kwon
University of California, Irvine
Irvine, CA, USA

{yongfal, hyoukjun.kwon}@uci.edu

Abstract

Stereo depth estimation is a fundamental component in augmented reality (AR), which requires low latency for real-time processing. However, preprocessing such as rectification and non-ML computations such as cost volume require significant amount of latency exceeding that of an ML model itself, which hinders the real-time processing required by AR. Therefore, we develop alternative approaches to the rectification and cost volume that consider ML acceleration (GPU and NPUs) in recent hardware. For pre-processing, we eliminate it by introducing homography matrix prediction network with a rectification positional encoding (RPE), which delivers both low latency and robustness to unrectified images. For cost volume, we replace it with a group-pointwise convolution-based operator and approximation of cosine similarity based on layernorm and dot product.

Based on our approaches, we develop MultiHeadDepth (replacing cost volume) and HomoDepth (MultiHeadDepth + removing pre-processing) models. MultiHeadDepth provides 11.8-30.3% improvements in accuracy and 22.9-25.2% reduction in latency compared to a state-of-the-art depth estimation model for AR glasses from industry. HomoDepth, which can directly process unrectified images, reduces the end-to-end latency by 44.5%. We also introduce a multi-task learning method to handle misaligned stereo inputs on HomoDepth, which reduces the AbsRel error by 10.0-24.3%. The overall results demonstrate the efficacy of our approaches, which not only reduce the inference latency but also improve the model performance. Our code is available at <https://github.com/UCI-ISA-Lab/MultiHeadDepth-HomoDepth>

1. Introduction

Depth estimation serves as a foundational component in augmented and virtual reality (AR/VR) [18] with many downstream algorithms, which include novel-view rendering [16, 31], occlusion reasoning [19], world locking for AR object placement [20], and determining the scale of AR objects [3]. In the AR domain, stereo depth estimation is often

deployed [31] rather than mono depth estimation due to its superior accuracy and the ease of deploying stereo cameras on the each side of AR glasses frames. Due to the realtime nature of AR applications [18], one key objective for depth estimation models for AR is low latency, in addition to good performance. However, since AR glasses are in a compute resource-constrained wearable form factor, enabling desired latency (less than 100 ms on device) is not trivial.

One challenge towards the latency optimization originates from the preprocessing, which is a indispensable step for achieving high model performance in practical applications. Examples include camera calibration and rectification using camera intrinsic and extrinsic. In addition to preprocessing, traditional algorithms embedded into depth estimation models (e.g, cost volume [8, 31]) also imposes another challenge to the latency optimization. We present their significance in latency in Fig. 1, which shows that preprocessing accounts for 30.2% and cost volume accounts for 29.3% of the total latency of a state-of-the-art (SOTA) model, ARGOS [31]. Note that preprocessing latency can be more dominant when camera intrinsic and extrinsic parameters are unknown, which results in 200 to 2000 ms latency to solve for the extrinsic parameters and subsequently process the images [17, 34]. This aggravates the long preprocessing latency challenge further.

Therefore, to reduce depth estimation model latency for AR glasses, we propose new methodologies that significantly reduce the preprocessing (online stereo rectification not required) and cost volume latency, as shown in Fig. 1. Our approach for the cost volume is (1) to replace traditional algorithm with group-pointwise convolutions, which are highly optimized in hardware and compilers and (2) adopt an efficient approximation of cosine similarity using layernorm and dot product. For preprocessing (stereo matching), we adopt homography matrix-based approximation and estimate homography using a head attached to the depth estimation model, which allows to utilize homography matrix-based approximation with dynamically varying extrinsic parameters in unstable AR glasses platform or no access to camera extrinsic parameters.

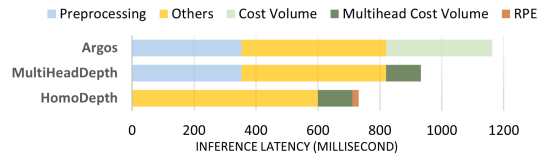


Figure 1. **The latency breakdown analysis of a SOTA model ARGOS [31] and ours, MULTIHEADDEPTH & HOMODEPTH** on Intel i7-12700H laptop CPU. The "RPE" refers to the 2D rectification position encoding process. "Others" refers to all the other parts of the neural network excluding cost volume blocks, such as Conv, Norm, FC, and ReLU6.

Since our methodologies are complementary to existing depth estimation models, we augment our methodologies on the ARGOS [31] and develop two new models, MULTIHEADDEPTH and HOMODEPTH. As presented in Fig. 1, MULTIHEADDEPTH focuses on cost volume optimization, which reduces inference latency by 25% compared to the original cost volume. HOMODEPTH targets scenarios that require stereo matching preprocessing, which can directly accept unrectified images as input, eliminating the needs for preprocessing. HOMODEPTH not only provides high-quality outputs on unrectified images as presented in Tab. 4, but also significantly reduces the end-to-end latency by 43%. Our evaluation includes ADT [22] dataset collected by real research AR glasses by Meta, Aria [6], which demonstrates the effectiveness of our approach on realistic AR glasses platform.

We summarize our contributions as follows:

- We develop MULTIHEADCOSTVOLUME block that replaces cost volume in a previous depth estimation model. Our new block provides significantly lower latency as well as higher accuracy.
- We introduce the homography of stereo images to reveal their position relationship to enable to accept unrectified images without preprocessing. We merge a small homography estimation head within the depth estimation network, which significantly reduces the latency compared to the preprocessing-based approach.
- We augment our homography estimation head with 2D rectification position encoding, which helps translate relative positional information from homography matrix to the 2D rectification position encoding format. It enables the neural network to effectively understand relative position information, which plays a key role in eliminating the need for rectification preprocessing.

2. Related Work

Stereo Depth Estimation. Fast depth estimation on mobile devices is a challenging task. Multiple previous works focused on delivering both model performance and efficiency targeting mobile platforms.

MOBILESTEREONET [27] is a stereo depth estimation model optimized for mobile devices, featuring a lightweight

network design and algorithms specially adapted for stereo matching. However, despite its minimal parameter size, the computational complexity is not small with high FLOPS (e.g. 190G FLOPS), which makes it challenging to achieve efficient performance on mobile platforms.

ARGOS [31] extended TIEFENRAUSCH monocular network [16] to implement stereo depth estimation. ARGOS is optimized to minimize the model size and computational complexity. ARGOS consists of an encoder, cost volume, and a decoder, which requires preprocessing if the input images are distorted or not rectified. ARGOS was designed for practical AR applications by Meta, and it delivers SOTA performance in the stereo depth estimation on AR glasses domain.

DYNAMICSTEREO [13] is a stereo depth estimation model designed for AR and mobile devices with a focus on efficiency, but ARGOS delivers superior accuracy and latency for just one-shot estimation.

Stereo Image Preprocessing. The stereo image preprocessing consists of calibration and rectification. Calibration is required for a raw image captured by the camera, which removes distortion and artifacts in the raw image. Calibration depends on the intrinsic parameters of camera systems. Generally, the intrinsic of a camera is stable and provided by manufacturers. Calibration can be completed with just a few matrix computations, within a few dozen milliseconds.

Rectification is a major challenge in stereo vision systems. For an object in the real world, its projection onto two image planes should lie on the epipolar lines, satisfying the epipolar constraint [7]. If the two cameras are misaligned, it is necessary to rectify the pair of images so that corresponding points lie on the same horizontal lines. The stereo cameras are mounted on both sides of AR glasses. The glasses may undergo significant bending ($> 10^\circ$) because of the soft material [31]. This issue can be exacerbated by variations in users' head sizes and temperatures.

Rectification requires the relative position information of two cameras. MOBIDEPTH [34] assumed that the relative positions of the cameras had been obtained before the inference. However, practically, the camera positions on AR glasses need to be determined based on a pair of arbitrarily captured images, which requires online rectification unlike MOBIDEPTH's offline method. Karaev *et al.* [13] set image saturation to a value sampled uniformly between 0 and 1.4 during the model training phase to make the model more robust for stereo images misalignment. But it lacks good interpolation for robustness. Wang *et al.* proposed a fast online rectification [31]. However, there remains a 15% to 23% chance of failing to rectify the stereo images.

Every stereo-matching algorithm includes a mechanism for evaluating the similarity of image locations. Matching cost is calculated for every pixel over the range of examined disparities, and it's a commonly used measure-

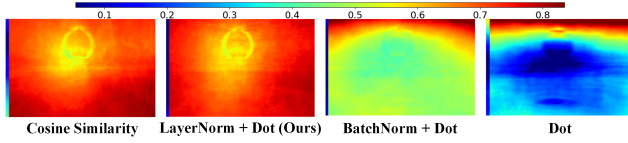


Figure 2. **Similarity estimation methodology Comparison.** Each represents a similarity map between left and right features after applying *roll* operation (offset: 10) to the input stereo images. The maps are average results across the entire dataset of SceneFlow [21], after rescaling to [0,1] range. The strips on the left side of maps are caused by *roll* and they are part of the maps.

ment [11]. Kendall *et al.* [14] adopted deep unary features to compute the stereo matching cost by forming a cost volume. They form a cost volume of dimensionality $height \times weight \times max_disparity \times features$ and pack it into a 4D volume. To construct the 4D volume, the algorithm iteratively move one of the images/features horizontally and compare the matching cost in each disparity, as described in Fig. 3a. The cost volume information is crucial for a neural network to understand the differences between the two feature maps. Although proven effective, the cost volume has a disadvantage in the computational efficiency from its cosine similarity computation described below:

$$D_{cos}(a, b) = a \cdot b / |a||b| \quad (1)$$

where a and b are the vector of pixels. The numerator is matrix multiplication-optimized hardware- and compiler-friendly since the dot product computation in a batch is a matrix multiplication. However, the denominator requires to compute norms pixel by pixel, which is not friendly to hardware highly optimized for matrix multiplication. Therefore, we first focus on the optimization of cost volume, which we discuss next.

3. MultiHeadDepth Model

Our approach to optimize cost volume is to replace it by a hardware-friendly approximation. We discuss our approach in detail and our solution, multi-head cost volume, that implements our approach.

3.1. Approximating Cosine Similarity

As discussed in Sec. 2, the denominator of cosine similarity (computing two norms and multiplication of them) and the corresponding division are the prime optimization targets. The goal of those operations is to normalize individual vectors to bound the results in the $[-1, 1]$ range. Based on the observation, we approximate the normalization for individual vectors using 2D layer normalization. This approach significantly reduces the number of costly normalization computations, as discussed in Appendix A. We still keep the numerator of cosine similarity as a dot product operator after the normalization. We term our approach as LND (Layer-Norm combined with Dot product). LND successfully approximates the cosine similarity, as shown by the results in

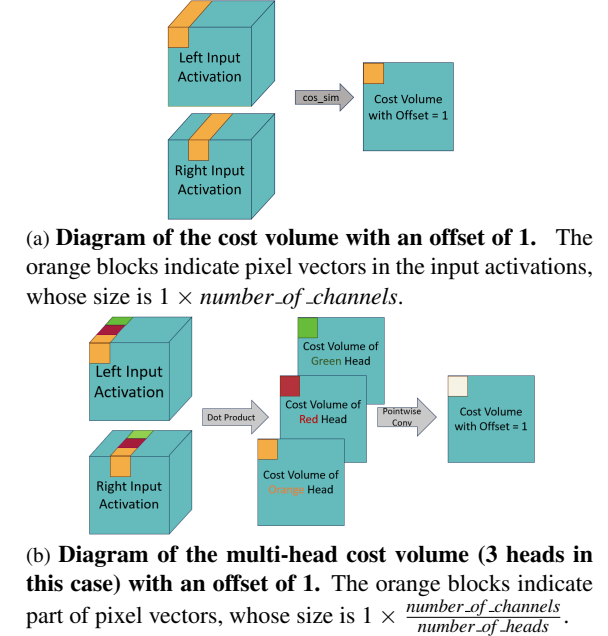


Figure 3. The comparison between the original cost volume and our approach, MULTIHEADCOSTVOLUME

Fig. 2. Another benefit of the layer norm is that it provides a buffer for the cost volume, which can accept additional encoded information. That is, each normalized pixel can be merged with other encodings. For instance, it can handle positional encoding, which we discuss in Sec. 4.4.

To further enhance the model performance, we also augment the LND with a weight layer after the LND, which help adjust the similarity approximation. Our evaluation results with better model performance and latency show the effectiveness of our approach that combines an efficient approximation and adaptive weight layers.

3.2. Multi-head Cost Volume

As another enhancement, we introduce the multi-head attention and dot scale [29] into our cost volume since separating the input activation in multiple heads enable more perceptions of the network. The approach also leads to a hardware-friendly operator, group-wise dot product followed by a point-wise convolution, as shown in Fig. 3 (b). This replaces the costly cosine similarity depicted in Fig. 3 (a), while providing better model performance. We describe the multi-head cost volume algorithm in a pseudo-code in Algorithm 1.

The benefits of multi-head cost volume can be summarized with (1) less computational complexity, (2) replacing the computation with highly-optimized group-pointwise Conv, and (3) potential accuracy improvement by providing more perceptions of the input activations.

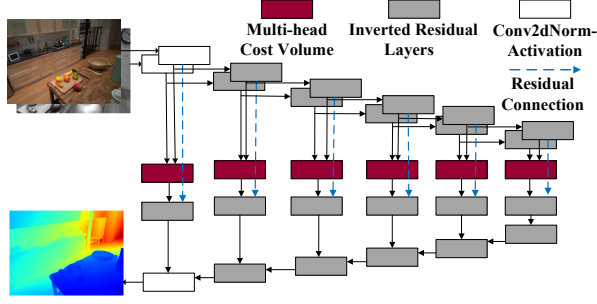


Figure 4. **The structure of MULTIHEADDEPTH.** The dashed lines indicate that the input activations from the left image are passed to the decoders. The input example is from ADT dataset.

Algorithm 1 Multi-head Cost Volume

Input:

$left : [NCHW], right : [NCHW], max_disparity : int,$
 $head_num : int \quad \triangleright \text{number of heads, can divide } C \text{ evenly}$

Output:

$cost_volume : [N, max_disparity, H, W]$

```

 $s = C/head\_num \quad \triangleright \text{stride of each head}$ 
 $left\_norm = \text{LayerNorm}(left)$ 
 $right\_norm = \text{LayerNorm}(right)$ 
for  $i \in 1 : max\_disparity + 1$  do
     $right\_shifted = \text{roll}(right\_norm, i) \quad \triangleright \text{roll: right-shifts the}$ 
     $\text{image by } i \text{ pixels, and the right-most } i \text{ columns move to the left}$ 
    for  $h \in 0 : head\_num$  do
         $left\_slice = left\_norm[:, hs : (h + 1)s, :, :]$ 
         $right\_slice = right\_shifted[:, hs : (h + 1)s, :, :]$ 
         $similarity[:, h, :, :] = \text{dot}(left\_slice, right\_slice)$ 
    end for
     $cost\_volume[:, i, :, :] = \text{pointwise\_conv}(similarity)$ 
end for

```

* In real code, the second loop and pointwise_conv are replaced by the memory operations and group-pointwise convolution

3.3. Structure of MULTIHEADDEPTH

Applying our approaches discussed in Sec. 3.1 and Sec. 3.2 to ARGOS [31], we design MULTIHEADDEPTH, which is illustrated in Fig. 4. All settings for the inverted residual blocks [25] in ARGOS are retained to highlight the efficacy of our Multi-head Cost Volume in an ablation study. Note that we apply our approach to Argos since it is the state-of-the-art today; our approach can be applied to any other depth estimation models with cost volume as well.

Upon this model, MULTIHEADDEPTH, we add an optimization targeting preprocessing, which we discuss next.

4. HomoDepth Model

We discuss how we augment MULTIHEADDEPTH with an optimization for preprocessing. We first introduce the 3D projection and homographic matrix and discuss how we estimate it using a homography estimation head.

4.1. 3D Projection & Homographic Matrix

The main functionality of cost volume is stereo-matching, which evaluates the similarity of left and right inputs with different offsets to determine the possible disparity. Because the algorithm only allows horizontal offsets, it can only match patterns on a common horizontal line. Therefore, preprocessing is required for the input images for stereo cameras without good alignments. We discuss depth estimation from a broader perspective, focusing on the projection relationship of a world point in the image planes of a stereo camera system. Aligned with that, we pose a fundamental question: **Given that the image points q_l and q_r are derived from the same world point Q , what is the relationship between q_l and q_r ?**

Suppose the distance from Q to the left and right image planes, intrinsic parameters, and extrinsic parameters are (d_l, d_r) , (K_l, K_r) , and (M_l, M_r) , respectively. Then the relationship between q_l and Q , q_r and Q is as follows:

$$q_l = \frac{1}{d_l} K_l M_l Q ; \quad q_r = \frac{1}{d_r} K_r M_r Q \quad (2)$$

Combining them, we obtain

$$q_r = \frac{d_l}{d_r} K_r M_r M_l^{-1} K_l^{-1} q_l = \frac{d_l}{d_r} H_{l \rightarrow r} q_l \quad (3)$$

where $H_{l \rightarrow r}$ is the 3×3 plane homography matrix [10], which converts q_l to q_r . That is, it is easy to represent the positional relationship between the content in stereo images if we know the homographic matrix and distance information.

Unfortunately, distance information is unknown in the depth estimation task since it is the eventual goal of the task. However, we can still leverage the following property: $d_l/d_r \approx 1$ when Q is on the central plane of the cameras or far from the cameras. Note that most imaged objects maintain a certain distance from the imaging system, in practice. Thus, homography matrix can approximately represent the positional relationship of stereo image planes, which is helpful for rectification. Homography is widely used in stereo vision applications, and for example, MVSNet [33] has demonstrated the effectiveness of homography.

MVSNet solves for homography under the assumption that the camera parameters are known but the depth is unknown. Unlike MVSNet, we target more challenging and practical scenarios where both camera extrinsic parameters and depth are unknown. We discuss our methodology to obtain the homography under such conditions next.

4.2. Homography Estimation Head

One challenge from unstable stereo vision systems like AR glasses is that extrinsic parameters keep changing. That is, we can not get the homography matrix directly from Eq. (3). As one solution, DeTone *et al.* [5] utilized a convolutional

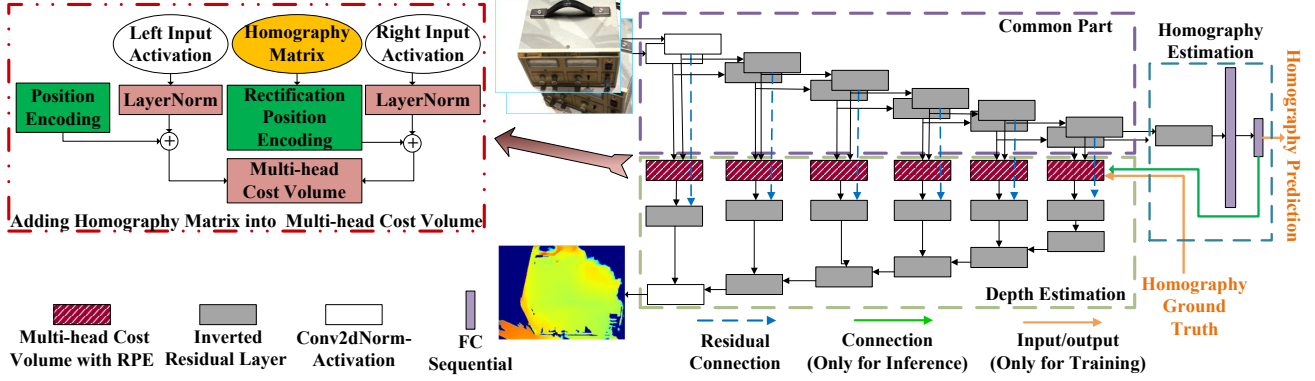


Figure 5. **Structure of HOMODEPTH.** The input example is from DTU dataset.

neural network (CNN) for homography estimation. Following the intuition, we also designed a CNN for estimating homography matrix. Unlike the previous work, which designed a stand-alone network dedicated for homography matrix estimation, our homography head shares the encoder with depth estimation, which is trained using a multi-task training technique discussed in Sec. 4.5.

4.3. 2D Rectification Positional Encoding (RPE)

Unlike classical epipolar rectification, or the fast online rectification proposed by Argos [31], homography rectification only needs to transfer one image for alignment. However, one challenge is the lost image information due to margins introduced in the rectified images.

Therefore, to eliminate the needs for homography rectification, we introduce a 2D rectification positional encoding (RPE) methodology that represents the position relationship between stereo images. This approach converts the homography matrix into positional encoding, which involves no information loss unlike homography rectification. Following [29], we define the 2D positional encoding (PE) as:

$$PE_i(q) = PE_i(x, y) = \begin{cases} \sin \frac{x}{f^{i/d}} & \text{if } i = 4k \\ \cos \frac{x}{f^{(i-1)/d}} & \text{if } i = 4k + 1 \\ \sin \frac{y}{f^{i/d}} & \text{if } i = 4k + 2 \\ \cos \frac{y}{f^{(i-1)/d}} & \text{if } i = 4k + 3 \end{cases} \quad (4)$$

$$i \in \{4k \leq i \leq 4k + 3 | i \in \mathbb{N}, k \in \mathbb{N}, k \in [0, C/4]\}$$

where q is a pixel located at coordinate (x, y) . C is the number of channels of input activation. i is the channel index. f is the encoding frequency. The value of f depends on the length of the sequence len or the size of the input activation. Generally, it should satisfy $2\pi f > len$, and we default as $f = 200$. Here $\mathbb{N}$ means the set of natural numbers.

Note that (x, y) are the coordinates of pixels. And Eq. (3) reveals the relationship between q_l and q_r . We follow Eq. (4) to get the positional encoding of q_l , noted as $PE(q_l)$. Then we apply 2D rectification positional encoding (RPE) to q_r as follows:

$$RPE(q_r) := PE(H_{l \rightarrow r} q_l) \quad (5)$$

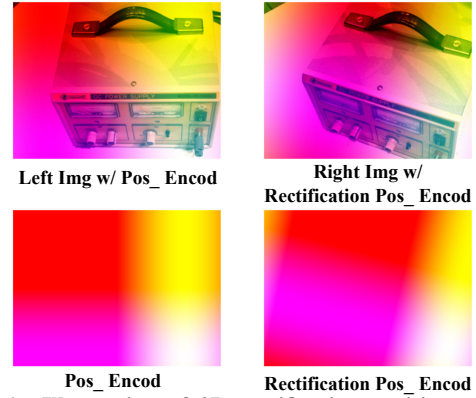


Figure 6. **Illustration of 2D rectification position encoding.** Color patterns indicate the values of positional encodings.

Because both $PE(q_l)$ and $RPE(q_r)$ use the same coordinates, **the same location in the real world leads to similar position encoding values in stereo images.** Without RPE, stereo matching solely relies on images semantic similarity. However, RPE incorporates positional similarity, which enhances the similarity across two image points from the same world point utilizing the additional positional information.

We present an example of positional encoding in Fig. 6 using colors representing regions with matched encoding values. The left and right images are obtained under a given homography using Eq. (4) and Eq. (5), respectively. We observe that the yellow pattern consistently appears on the right buckle of the leather strap, while the front edge of the power supply is always positioned between the red and pink patterns. The results show the efficacy of RPE in capturing alignment information in given stereo images or activations.

4.4. Structure of HOMODEPTH

Integrating our approaches discussed in Sec. 4.2 and Sec. 4.3 to MULTIHEADDEPTH, we develop HOMODEPTH, which is illustrated in Fig. 5. One key feature is the multi-task, which generates homography and depth estimation results using two heads connected to the encoder. Also, the RPE is integrated into the multi-head cost volume blocks and estimated homography matrix is used as an input to the multi-head cost volume block, as shown in Fig. 5.

Dataset	SceneFlow	ADT	DTU
Scenario	Synthetic scenarios (e.g. driving & flying items)	Ego-centric daily life scenarios	Robot arm rotating over a single item
Images Quality	Synthetic camera-based calibrated RGB images	Real camera-based RGB and Grayscale images	Real camera-based calibrated RGB images
Stereo Quality	Rectified	Rectified	Non-Rectified
Stereo Baseline	Static, 1.0 meter	Static, 12.8 cm	Dynamic, 7.8-14.9cm
Depth Map	Obtained by synthetic rendering	Obtained by twin digital synthetic rendering	Rendered from point clouds (need masks for invalid areas)

Table 1. Summary of Datasets

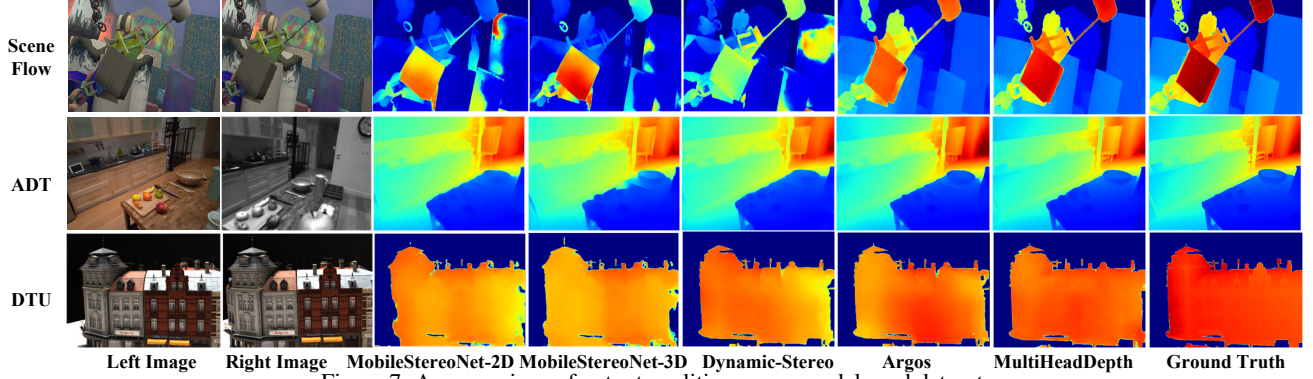


Figure 7. A comparison of output qualities across models and datasets

In Fig. 5, we highlight training and inference flows using orange and green arrows, respectively. In the inference phase, the predicted homography matrix is passed to all the multi-head cost volumes, and the RPE is calculated inside them. The range of positional encoding is between 0 and 1, and it is added to the layer norm output. This ensures that the weights of input activations and positional encoding are balanced in the cost volume, allowing similarity to be assessed using both pattern and positional information. In the training phase, the FC block and multi-head cost volume block are disconnected, resulting in two separate inputs and outputs in the model. The inputs include RGB stereo images and their homography matrix, while the outputs are the estimated homography and depth.

Overall, the model structure is based on three components: a common part (encoder), a depth estimation head, and a homography estimation head, as shown in Fig. 5. The shared encoder across two heads reduces overall computational costs and enables to perform two tasks with only a minor increase in latency for extra homography estimation head. However, training a multi-task model like ours requires a specialized approach, which we discuss next.

4.5. Multi-task Model Training

To enable multi-task training of HOMODEPTH, we utilize the homoscedastic uncertainty [15] to train the model with two loss functions. The first loss function of depth estimation is adopted from ARGOS [31],

$$L_D(y, \hat{y}) = SL_1(y, \hat{y}) + \sum_{l=0}^4 SL_1(\nabla y^l, \nabla \hat{y}^l) \quad (6)$$

where SL_1 is the SmoothL1Loss [9]. ∇y^l indicates the gradient of the depth map which is 2×2 subsampled l times. y is the ground truth, and $\hat{y}$ is the prediction.

The second loss function of homography estimation is,

$$L_H(y, \hat{y}) = ||weight_w(y) - weight_w(\hat{y})||_F \quad (7)$$

where

$$weight_w(y) = \begin{pmatrix} w & w & 1 \\ w & w & 1 \\ 1 & 1 & w \end{pmatrix} \odot y \quad (8)$$

In the homography matrix, the first two rows and two columns represent the angular relationship between planes, usually with smaller values. The last column of the first two rows indicates the distance relationship between planes, generally with larger values. The $weight$ function applies weighting to smaller values by element-wise production, ensuring that all elements of the homography matrix are perceptible in the loss function, which is helpful with better homography matrix estimations. By default, we set w as 50.

Utilizing the loss functions for homography and depth estimation, we formulate combined loss function as:

$$L = \frac{L_H}{2\sigma_H^2} + \frac{L_D}{2\sigma_D^2} + \log \sigma_H \sigma_D \quad (9)$$

where σ_H and σ_D represent homoscedastic uncertainties of two tasks. They are trainable parameters in HOMODEPTH.

5. Evaluation

We evaluate the effectiveness and efficiency of MULTI-HEADDEPTH and HOMODEPTH using three datasets on three platforms. We also show the quantization results of the models and discuss the robustness of HOMODEPTH.

5.1. Datasets

We utilize SceneFlow [21], DTU Robot Image Datasets (DTU) [12], and Aria Digital Twin (ADT) [22] datasets to

	MobileStereoNet-2D (WACV 2022 [27])			MobileStereoNet-3D (WACV 2022 [27])			Dynamic-Stereo (CVPR 2023 [13])			Argos (CVPR 2023 [31])			Selective-Stereo (CVPR 2024 [32])			MultiheadDepth (Ours)		
	AbsRel	D1	RMSE	AbsRel	D1	RMSE	AbsRel	D1	RMSE	AbsRel	D1	RMSE	AbsRel	D1	RMSE	AbsRel	D1	RMSE
SceneFlow	0.172	0.71	8.2	0.129	0.42	5.9	0.287	0.89	68.8	0.102	0.23	5.3	0.053	0.07	21.99	0.091	0.35	4.3
Middlebury	0.180	0.43	65.6	0.139	0.32	25.7	0.151	0.38	20.15	0.107	0.016	27.54	0.169	0.41	12.90	0.094	0.01	23.6
ADT	0.199	0.36	611.9	0.135	0.52	563.3	0.176	0.55	546.1	0.133	0.34	320.1	0.082	0.25	277.3	0.094	0.25	273.6
DTU	0.147	0.38	315.9	0.148	0.40	224.8	0.339	0.88	628.5	0.122	0.61	536.9	0.128	0.49	229.7	0.101	0.42	216.1

Table 2. **The overall accuracy of the models.** The datasets are used as inputs without any preprocessing. As indicated in Tab. 1, SceneFlow, Middlebury, and ADT datasets consist of rectified stereo image inputs, while DTU contains unrectified stereo image inputs. For Middlebury, following the practice in [31], models are trained with SceneFlow and tested on Middlebury 2014.

	AbsRel of Datasets			Latency (ms)	
	SceneFlow	ADT	DTU	CPU	GPU
Argos (CVPR 2023)	0.109	0.146	0.140	748.5	54.4
MultiheadDepth (Ours)	0.098	0.097	0.112	598.9	45.3

Table 3. **The performance of quantized models (INT8).** The latencies are measured on a laptop with Intel® Core™ i7-12700H CPU and Nvidia® GeForce RTX 3070 Ti laptop GPU.

evaluate our models and a state-of-the-art stereo depth estimation model for AR glasses, Argos [31]. We summarize the characteristics of the datasets in Tab. 1.

SceneFlow SceneFlow is widely used for depth estimation tasks. However, the dataset contains rendered images from synthetic scenarios, which does not fully represent real-world scenarios. Also, its baseline is one meter, which is not aligned with that of AR glasses.

Aria Digital Twin (ADT) ADT consists of images captured by RGB and gray-scale fisheye cameras mounted on Aria AR glasses [6], prototype AR glasses developed by Meta. Accordingly, the scenarios in the dataset are designed for AR applications. However, the ground truth depth ignores human body in the images. To avoid disturbance from the property, we select 145 out of 236 scenarios that do not involve any human in the scene. Due to Aria AR glasses’ hardware configuration, each scene in ADT consists of an RGB image from the left and a gray-scale fisheye image from the right. Since both RGB and fisheye images are distorted, we apply calibration before providing images to the models. Also, we replicate the calibrated grayscale images on channel dimension to match the shape of the RGB input tensor. We discuss more details about ADT in Appendix C.

DTU Robot Image Dataset (DTU) DTU dataset is designed for multi-view stereo (MVS) reconstruction task. The dataset consists of images captured by a camera mounted on a robotic arm performing spherical scans, which moves the robot arm from left to right with a slight downward tilt to the left back upon reaching the edge. We utilize images captured at adjacent positions as stereo image pairs for training. Compared to other MVS datasets, the DTU dataset is closer to the AR glasses scenarios since (1) its scanning trajectory is close to the horizontal line, which can mimic the misalignment of stereo images. (2) The scanning intervals between adjacent images (i.e., baselines), are ≈ 10 cm, which is similar as the baseline of AR glasses [6].

We train our model for the three datasets individually and evaluate the performance. Since the DTU contains the extrinsic parameters of each frame, we leverage them in calcu-

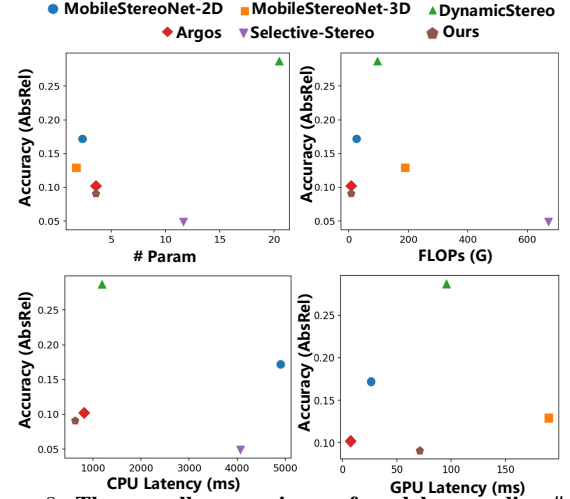


Figure 8. **The overall comparisons of models regarding # parameters, FLOPs, and latencies versus accuracy.** For all coordinates, the closer to the bottom left, the better model performance. Here CPU indicates Core™ i7-12700H and GPU indicates Nvidia® GeForce RTX 3070 Ti laptop GPU.

lating the homography matrix over each stereo image inputs taken from two adjacent frames. We utilize the calculated homography matrix as the ground truth for the homography estimation in HOMODEPTH. In the accuracy evaluation, we also evaluate our model on Middlebury 2014 [26].

5.2. Implementation details

We implement MULTIHEADDEPTH and HOMODEPTH in Pytorch [23] and train on Nvidia RTX 4090 24GB GPU. We resize all input images in all experiments to $288 \times 384 \times 3$. We use Adam optimizer without schedulers. In the early stages of model training, the base learning rate (LR) is $1e-4$, and we select the best epoch. Afterward, we fine-tune the model with an LR of $4e-4$ and select the optimal weights. We set the batch size as 10, based on the GPU memory.

5.3. Evaluation Metrics and Platform

We adopt AbsRel, D1, and RMSE as the evaluation metrics to measure model performance. All the metrics are "Lower is Better" metrics, where smaller values represent higher model performance. We provide the detailed definition of each metric in Appendix D.

Since our models’ eventual target is AR glasses, and there is no AR glasses with open API to the best of our

Dataset		Argos			PreP+Argos			PreP+MultiHeadDepth			HomoDepth		
		AbsRel	D1	RMSE	AbsRel	D1	RMSE	AbsRel	D1	RMSE	AbsRel	D1	RMSE
DTU	DTU	0.122	0.61	536.91	0.109	0.54	210.8	0.099	0.38	204.4	0.098	0.38	208.8
	DTU_df	0.183	0.64	599.32	0.115	0.55	223.1	0.107	0.48	287.8	0.087	0.32	184.9
	Sceneflow_persp	0.232	0.71	597.33	0.121	0.43	209.9	0.114	0.42	215.3	0.097	0.40	198.4
Latency (ms)	CPU	811.0			1068.3			884.4			761.2		
	GPU	109.0			312.5			312.6			84.5		

Table 4. **Ablation study comparing models with or without rectification preprocessing (PreP).** DTU dataset represents a stereo system with a fixed focal length but variable relative position, while DTU_df represents a system with both variable focal length and position. Sceneflow_persp is the simulation of glasses bending by perspective transformation.

knowledge, we use three mobile platforms for latency evaluation as proxy. For the main model performance evaluation, we use a laptop with Intel® Core™ i7-12700H CPU and Nvidia® GeForce RTX 3070 Ti laptop GPU. We also measure the inference latency of our models on Nvidia® Jetson Orin Nano™ Developer Kit [1] and a smartphone equipped with Snapdragon 8+ Gen 1 [2].

5.4. MULTIHEADDEPTH Performance

We first evaluate the model performance on calibrated inputs. We compare the accuracy of MULTIHEADDEPTH against SOTA stereo depth estimation models listed in Tab. 2. The overall results plotted in Fig. 8 demonstrate that ARGOS and MULTIHEADDEPTH are superior solutions for evaluated scenarios designed to model AR glasses. Comparing MULTIHEADDEPTH against ARGOS, MULTIHEADDEPTH provides 11.8-30.3% improvements in accuracy and 22.9-25.2% reduction in latency. Compared to Selective-Stereo, MULTIHEADDEPTH achieves competitive accuracy while using only 1.2% of the FLOPs required by Selective-Stereo. The results show the effectiveness of our approach to optimize the cost volume blocks.

Impact of Quantization As quantization is commonly adopted optimization like Argos [31], we also evaluate quantized models and present the results in Tab. 3. In this study, we use Aimet [28] from Qualcomm to perform a post-training quantization. Although the overall accuracy decreases slightly by 3.1-14.7%, the latency is considerably reduced by 7.7-43.4% after quantization. MULTIHEADDEPTH remains superior in all metrics after quantization.

5.5. HOMODEPTH Performance

Since HOMODEPTH optimizes preprocessing, we focus on the DTU scenarios that require preprocessing (rectification) for evaluating HOMODEPTH. Utilizing the DTU dataset, we create a custom dataset, DTU_df, by scaling and cropping DTU images to simulate varying focal lengths ("df": dynamic focal length). Thus, the DTU dataset represents a stereo system with a fixed focal length but variable relative position, while DTU_df represents a system with both variable focal length and relative position. Furthermore, we apply perspective transformation on Sceneflow (Sceneflow_persp) to simulate the misalignment of cameras.

In Tab. 4, we evaluate four models with and without preprocessing. Although MULTIHEADDEPTH requires less in-

	Orin Nano		Snapdragon	
	CPU	GPU	CPU	GPU
Argos (CVPR 2023)	8774	209	1424	914
MultiHeadDepth (Ours)	6183	216	1156	617
HomoDepth (Ours)	6611	203	1512	893

Table 5. **Latency of models in millisecond** on NVIDIA® Jetson Orin Nano™ Developer Kit and Snapdragon 8+ Gen 1 Platform.

ference latency than HOMODEPTH, it still relies on preprocessing to achieve better accuracy, as shown in Tab. 2 and Tab. 4. In contrast, HOMODEPTH does not require preprocessing. When the stereo cameras are misaligned, HOMODEPTH becomes a better solution, delivering higher accuracy with 30% less end-to-end latency.

5.6. Latency on Edge Devices

We report the latencies on two mobile/edge scale devices, Nvidia® Jetson Orin Nano™ Developer Kit and Snapdragon 8+ Gen 1 Mobile Platform in Tab. 5. On the Orin Nano, the models are executed in eager mode. On the Snapdragon, the models are compiled using SNPE [24] and executed via ADB commands. The results may vary due to different deployment methods. We observe our models overall achieve better latency compared to Argos. Note that Argos results do not include preprocessing delay, while HOMODEPTH can accept unrectified data without preprocessing.

6. Conclusion

Achieving both high model performance and low latency is a key toward depth estimation on AR glasses. In this work, we identify cost volume and preprocessing as the major optimization targets for low latency. For cost volume, we develop a hardware-friendly approximation of cosine similarity and employ a group-pointwise convolution. For preprocessing, we introduce rectification positional encoding (RPE) and leverage homography matrix estimated by a head attached to the common encoder in HOMODEPTH, which significantly reduces the computational complexity.

Our approaches are complementary to any stereo depth estimation models with online rectification and/or cost volume. In addition to the broad applicability, our evaluation results show the effectiveness of our approaches in model performance as well as latency. Therefore, we believe our work is making a meaningful step forward in the field of stereo depth estimation systems for AR.

7. Acknowledgment

We thank Dilin Wang, Wei Ye, and Liangzhen Lai for their insightful feedback, Jialiang Wang for the guidance in modeling Argos, Meta for providing Aria AR glasses, and Rachid Karami for the assistance with the latency analysis.

References

- [1] Jetson orin nano developer kit getting started guide. <https://developer.nvidia.com/embedded/learn/get-started-jetson-orin-nano-devkit>. Accessed on Nov 12, 2024. 8
- [2] Snapdragon 8+ gen 1 mobile platform. <https://www.qualcomm.com/products/mobile/snapdragon/smartphones/snapdragon-8-series-mobile-platforms>. Accessed on Nov 12, 2024. 8
- [3] Jong-gil Ahn, Euijai Ahn, Seulki Min, Hyeonah Choi, Howon Kim, and Gerard J. Kim. Size perception of augmented objects by different ar displays. In *HCI International 2019 - Posters*, pages 337–344, Cham, 2019. Springer International Publishing. 1
- [4] PyTorch Contributors. *Document of torch.nn.LayerNorm*. Accessed on Mar 09, 2025. 11
- [5] Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Deep image homography estimation, 2016. 4
- [6] Jakob Engel, Kiran Somasundaram, Michael Goesele, Albert Sun, Alexander Gaminio, Andrew Turner, Arjang Talattof, Arnie Yuan, Bilal Souti, Brighid Meredith, et al. Project aria: A new tool for egocentric multi-modal ai research. *arXiv preprint arXiv:2308.13561*, 2023. 2, 7
- [7] Olivier Faugeras. *Three-dimensional computer vision: a geometric viewpoint*. MIT press, 1993. 2
- [8] Wanshui Gan, Pak Kin Wong, Guokuan Yu, Rongchen Zhao, and Chi Man Vong. Light-weight network for real-time adaptive stereo depth estimation. *Neurocomputing*, 441:118–127, 2021. 1
- [9] Ross Girshick. Fast r-cnn. In *2015 IEEE International Conference on Computer Vision (ICCV)*, pages 1440–1448, 2015. 6
- [10] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, New York, NY, USA, 2 edition, 2003. 4
- [11] Heiko Hirschmüller and Daniel Scharstein. Evaluation of cost functions for stereo matching. In *2007 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8, 2007. 3
- [12] Rasmus Jensen, Anders Dahl, George Vogiatzis, Engil Tola, and Henrik Aanæs. Large scale multi-view stereopsis evaluation. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pages 406–413. IEEE, 2014. 6
- [13] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Dynamicstereo: Consistent dynamic depth from stereo videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13229–13239, 2023. 2, 7
- [14] Alex Kendall, Hayk Martirosyan, Saumitro Dasgupta, Peter Henry, Ryan Kennedy, Abraham Bachrach, and Adam Bry. End-to-end learning of geometry and context for deep stereo regression. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017. 3
- [15] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7482–7491, 2018. 6
- [16] Johannes Kopf, Kevin Matzen, Suhil Alsian, Ocean Quigley, Francis Ge, Yangming Chong, Josh Patterson, Jan-Michael Frahm, Shu Wu, Matthew Yu, et al. One shot 3d photography. *ACM Transactions on Graphics (TOG)*, 39(4): 76–1, 2020. 1, 2
- [17] Sanjeev Kumar, Christian Micheloni, Claudio Piciarelli, and Gian Luca Foresti. Stereo rectification of uncalibrated and heterogeneous images. *Pattern Recognit. Lett.*, 31:1445–1452, 2010. 1
- [18] Hyoukjun Kwon, Krishnakumar Nair, Jamin Seo, Jason Yik, Debabrata Mohapatra, Dongyuan Zhan, Jinook Song, Peter Capak, Peizhao Zhang, Peter Vajda, et al. Xrbench: An extended reality (xr) machine learning benchmark suite for the metaverse. *Proceedings of Machine Learning and Systems*, 5, 2023. 1
- [19] Mengdi Li, Cornelius Weber, Matthias Kerzel, Jae Hee Lee, Zheni Zeng, Zhiyuan Liu, and Stefan Wermter. Robotic occlusion reasoning for efficient object existence prediction. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2686–2692. IEEE, 2021. 1
- [20] Hope Lutwak, T. Scott Murdison, and Kevin W. Rio. User Self-Motion Modulates the Perceptibility of Jitter for World-locked Objects in Augmented Reality. In *2023 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 346–355, 2023. 1
- [21] N. Mayer, E. Ilg, P. Häusser, P. Fischer, D. Cremers, A. Dosovitskiy, and T. Brox. A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation. In *IEEE International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. arXiv:1512.02134. 3, 6
- [22] Xiaqing Pan, Nicholas Charron, Yongqian Yang, Scott Peters, Thomas Whelan, Chen Kong, Omkar Parkhi, Richard Newcombe, and Yuheng (Carl) Ren. Aria digital twin: A new benchmark dataset for egocentric 3d machine perception. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 20133–20143, 2023. 2, 6
- [23] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA, 2019. 7
- [24] Inc. Qualcomm Technologies. *Snapdragon Neural Processing Engine (SNPE) Developer Guide*, 2024. 8
- [25] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018. 4
- [26] Daniel Scharstein, Heiko Hirschmüller, York Kitajima, Greg Krathwohl, Nera Nešić, Xi Wang, and Porter West-

- ling. High-resolution stereo datasets with subpixel-accurate ground truth. In *Pattern Recognition*, pages 31–42, Cham, 2014. Springer International Publishing. 7
- [27] Faranak Shamsafar, Samuel Woerz, Rafia Rahim, and Andreas Zell. Mobilestereonet: Towards lightweight deep networks for stereo matching. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 2417–2426, 2022. 2, 7
- [28] Sangeetha Siddegowda, Marios Fournarakis, Markus Nagel, Tijmen Blankevoort, Chirag Patel, and Abhijit Khobare. Neural network quantization with ai model efficiency toolkit (aimet). *arXiv preprint arXiv:2201.08442*, 2022. 8
- [29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 3, 5
- [30] Phaneendra Vinukonda. A study of the scale-invariant feature transform on a parallel pipeline. Master’s thesis, Louisiana State University, 2011. LSU Master’s Theses, 2721. 11
- [31] Jialiang Wang, Daniel Scharstein, Akash Bapat, Kevin Blackburn-Matzen, Matthew Yu, Jonathan Lehman, Suhil Alsisan, Yanghan Wang, Sam Tsai, Jan-Michael Frahm, et al. A practical stereo depth system for smart glasses. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21498–21507, 2023. 1, 2, 4, 5, 6, 7, 8, 11
- [32] Xianqi Wang, Gangwei Xu, Hao Jia, and Xin Yang. Selective-stereo: Adaptive frequency information selection for stereo matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19701–19710, 2024. 7
- [33] Yao Yao, Zixin Luo, Shiwei Li, Tian Fang, and Long Quan. Mvsnet: Depth inference for unstructured multi-view stereo. In *Proceedings of the European conference on computer vision (ECCV)*, pages 767–783, 2018. 4, 11
- [34] Jinrui Zhang, Huan Yang, Ju Ren, Deyu Zhang, Bangwen He, Ting Cao, Yuanchun Li, Yaoxue Zhang, and Yunxin Liu. Mobidepth: real-time depth estimation using on-device dual cameras. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, page 528–541, New York, NY, USA, 2022. Association for Computing Machinery. 1, 2